

Laptop Price Predictor -- Detailed Study and Interview Notes

1. Problem Statement

The project predicts the selling price of a laptop using its technical specifications. It is a supervised machine learning regression problem because the output is a continuous numerical value: laptop price.

The original dataset contains 1,303 records and 12 columns:

- Company
- TypeName
- Inches
- ScreenResolution
- Cpu
- Ram
- Memory
- Gpu
- OpSys
- Weight
- Price
- Unnamed: 0

The notebook transforms the original raw columns into a cleaner modelling dataset containing 12 input features and one target.

2. Data Preparation

2.1 Cleaning text-based numerical columns

Raw laptop datasets often store values with units:

- RAM may be stored as 8GB.
- Weight may be stored as 1.37kg.
- Memory may be stored as 128GB SSD.

These fields need to be converted into useful numerical or categorical representations before modelling.

2.2 Memory feature engineering

The notebook extracts storage categories such as HDD and SSD. The final modelling dataset includes:

- HDD
- SSD

The original detailed memory text is removed from the final feature set.

2.3 GPU processing

The notebook extracts the GPU brand from the full GPU description and removes the original detailed GPU text. It also removes records with the ARM GPU category.

2.4 Operating-system grouping

The notebook maps operating systems into three groups:

```
def cat_os(inp):
    if inp == "Windows 10" or inp == "Windows 7" or inp == "Windows 10 S":
        return "Windows"
    elif inp == "macOS" or inp == "Mac OS X":
        return "Mac"
    else:
        return "Others/No OS/Linux"
```

This reduces the number of categories and makes the feature easier to use.

2.5 PPI feature

The notebook calculates pixel density:

```
ppi = np.sqrt(x_resolution**2 + y_resolution**2) / screen_size
```

PPI can capture the relationship between resolution and physical display size.

3. Target Transformation

The notebook uses:

```
x = df.drop(columns=["Price"])
y = np.log(df["Price"])
```

This means the model learns to predict the natural logarithm of price.

During deployment:

```
prediction = pipe.predict(query)[0]
predicted_price = np.exp(prediction)
```

The exponential function reverses the logarithmic transformation.

Why use log-price?

Laptop prices may be right-skewed: many laptops have moderate prices while a smaller number of premium laptops are very expensive. Log transformation can:

- Reduce right skewness.
- Make extreme values less dominant.
- Help the model learn relative price differences.
- Sometimes improve residual behaviour.

A limitation is that evaluation performed directly on `y_test` and `y_pred` measures error in log-price space.

4. Train/Test Split

The notebook uses:

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.15,
    random_state=2
)
```

The resulting split contains:

- Training data: 1,106 rows
- Test data: approximately 197 rows
- Test proportion: 15%
- Random seed: 2

The test set is used to estimate how the trained model performs on unseen records.

5. Categorical Encoding

The notebook automatically identifies categorical columns:

```
categorical_cols = X_train.select_dtypes(
    include=["object", "category"]
).columns.tolist()
```

The categorical columns recorded in the notebook are:

- Company
- TypeName
- Cpu brand
- Gpu brand
- os

They are processed using:

```
OneHotEncoder(
    sparse_output=False,
    drop="first",
    handle_unknown="ignore"
)
```

Meaning of the parameters

- `sparse_output=False`: returns a dense numerical array.
- `drop="first"`: removes one category per feature to reduce redundancy.
- `handle_unknown="ignore"`: prevents errors for categories not observed during training.

The `ColumnTransformer` applies encoding only to categorical columns and passes numerical columns through unchanged.

6. Pipeline

The project uses a Scikit-learn Pipeline:

```
pipe = Pipeline([
    ("preprocessor", step1),
    ("regressor", step2)
])
```

The pipeline combines:

1. Feature transformation.
2. Model fitting.
3. Prediction.

This is important because the same fitted encoder must be used when the Streamlit app receives new user input.

7. Models and How They Work

7.1 Linear Regression

Linear Regression estimates a relationship between input features and

the target using a weighted sum.

Strengths:

- Simple and interpretable.
- Fast to train.
- Useful as a baseline.

Limitations:

- May not capture complex nonlinear relationships.

Recorded result:

- R^2 : 0.8073
- MAE: 0.2102 log units

7.2 Ridge Regression

Ridge is linear regression with L2 regularization. It discourages very large coefficients and can improve stability when many one-hot encoded features exist.

Recorded result:

- R^2 : 0.8127
- MAE: 0.2093 log units

7.3 Lasso Regression

Lasso uses L1 regularization. It can shrink some coefficients to zero and perform a form of feature selection.

Recorded result:

- R^2 : 0.8072
- MAE: 0.2111 log units

7.4 KNN Regression

KNN predicts a value based on nearby training observations. Scaling is useful because KNN relies on distances.

Recorded result:

- R^2 : 0.8543
- MAE: 0.1834 log units

7.5 Decision Tree

A Decision Tree divides the feature space into regions using decision rules. It can capture nonlinear relationships and interactions.

Parameter:

```
DecisionTreeRegressor(max_depth=8, random_state=42)
```

Recorded result:

- R^2 : 0.8423
- MAE: 0.1806 log units

7.6 Support Vector Regression

SVR attempts to fit a function while allowing a margin of error controlled by epsilon. The RBF kernel can model nonlinear patterns.

Parameters:

```
SVR(kernel="rbf", C=100, epsilon=0.1)
```

Recorded result:

- R^2 : 0.8574
- MAE: 0.1836 log units

7.7 Random Forest

Random Forest averages predictions from multiple decision trees. Each tree is trained using random sampling and feature randomness.

Parameters:

```
RandomForestRegressor(  
    n_estimators=100,  
    max_depth=15,  
    random_state=42,  
    n_jobs=-1  
)
```

Recorded result:

- R^2 : 0.8831
- MAE: 0.1583 log units

7.8 Extra Trees

Extra Trees is an ensemble of highly randomized decision trees. It introduces more randomness in split selection than Random Forest.

Recorded result:

- R^2 : 0.8701
- MAE: 0.1627 log units

7.9 AdaBoost

AdaBoost builds weak learners sequentially and focuses more on observations that previous learners handled poorly.

The notebook uses shallow decision trees as base estimators.

Recorded result:

- R^2 : 0.8054
- MAE: 0.2220 log units

7.10 Gradient Boosting

Gradient Boosting builds trees sequentially. Each new tree attempts to reduce the loss remaining from earlier trees.

Parameters:

```
GradientBoostingRegressor(  
    n_estimators=100,  
    learning_rate=0.05,  
    max_depth=4,  
    random_state=42  
)
```

Recorded result:

- R^2 : 0.8686
- MAE: 0.1751 log units

7.11 XGBoost

XGBoost is an optimized gradient-boosting algorithm designed for strong performance on structured/tabular data. It adds regularization and implementation optimizations.

Parameters used:

```
XGBRegressor(  
    n_estimators=100,  
    learning_rate=0.05,  
    max_depth=6,  
    subsample=0.8,  
    colsample_bytree=0.8,  
    objective="reg:squarederror",  
    random_state=42,  
    n_jobs=-1  
)
```

Recorded result:

- R^2 : 0.8964
- MAE: 0.1532 log units

This was the highest recorded R^2 score in the notebook results.

7.12 Voting Regressor

Voting Regressor averages predictions from:

- Random Forest
- Gradient Boosting
- Extra Trees

Recorded result:

- R^2 : 0.8869
- MAE: 0.1599 log units

7.13 Stacking Regressor

Stacking uses predictions from base regressors as inputs to a final estimator.

Base models:

- Random Forest
- Gradient Boosting
- Extra Trees

Final estimator:

```
Ridge(alpha=10)
```

Cross-validation:

```
cv=5
```

Recorded result:

- R^2 : 0.8868
- MAE: 0.1594 log units

The notebook then exports this final `pipe` variable, so the exported `pipe.pkl` corresponds to the last pipeline created in the notebook: the Stacking Regressor pipeline.

8. MAE in Detail

The formula is:

$$\text{MAE} = (1/n) \times \sum |\text{actual value} - \text{predicted value}|$$

MAE treats every absolute error equally and is easy to understand.

Why does MAE occur?

A model rarely predicts every record exactly because:

- The dataset may not contain all price-related features.
- Brand reputation and build quality may not be fully represented.
- The same RAM or CPU category can exist in different laptop product tiers.
- The dataset may contain noise or outdated prices.
- The model is trained on a limited number of examples.
- Some categories may have few observations.
- The selected algorithm may underfit or overfit.

Log-space warning

Because the notebook uses `y = np.log(Price)`, the MAE values printed during training are log-space errors. To report an understandable rupee-based MAE, transform both predictions and actual values back to rupees before calculating MAE.

9. R² in Detail

R² compares the model against a baseline that always predicts the average target value.

A high R² indicates that the model explains much of the variation in the target for the evaluated test set. It does not guarantee that all predictions are close in rupees.

Why can R² be less than 1?

Even a good model will have errors due to:

- Missing explanatory features.
- Noise in the data.

- Limited training examples.
- Incorrect or simplified feature categories.
- Different pricing strategies between manufacturers.
- Product condition, discounts, region, and availability not being represented.

Why can R^2 differ across algorithms?

Different algorithms have different inductive biases:

- Linear models assume a mainly linear relationship.
- Trees capture nonlinear rules.
- Random Forest reduces variance by averaging trees.
- Boosting models correct earlier errors sequentially.
- Ensemble models combine multiple prediction patterns.

10. Model Selection

The notebook recorded the strongest R^2 result for XGBoost:

- $R^2 \approx 0.8964$
- MAE ≈ 0.1532 in log-price space

However, the exported model is created from the final `pipe` variable, which is overwritten during the notebook and ends with the Stacking Regressor pipeline. Therefore:

- The best recorded model result is XGBoost.
- The exported model in the notebook is Stacking Regressor.
- The deployed model should be identified by inspecting the actual saved `pipe.pkl`.

This distinction is important in interviews. Never claim that the deployed application uses XGBoost unless the saved pipeline has been verified.

11. Streamlit Application Logic

The app loads the model and processed dataframe using file paths relative to the application file:

```
BASE_DIR = Path(__file__).resolve().parent
```

This is more reliable than depending on the current terminal directory.

The app collects user inputs and converts them into a Pandas DataFrame with the exact feature names expected by the pipeline.

PPI is calculated from resolution and screen size. The pipeline then performs encoding and prediction.

The predicted log-price is converted back:

```
predicted_price = np.exp(prediction)
```

12. Docker Explanation

The Dockerfile is:

```
FROM python:3.9
WORKDIR /app
COPY . /app
RUN pip install -r requirements.txt
EXPOSE 8501
CMD ["streamlit", "run", "app.py", "--server.address=0.0.0.0", "--server.port=8501"]
```

Instruction-by-instruction explanation

- FROM: chooses the base Python environment.
- WORKDIR: sets the working directory inside the image.
- COPY: copies project files into the image.
- RUN: installs Python dependencies.
- EXPOSE: documents the Streamlit port.
- CMD: starts the Streamlit server.

Docker commands

Build:

```
docker build -t laptop-price-predictor .
```

Run:

```
docker run -p 8501:8501 laptop-price-predictor
```

Open:

```
http://localhost:8501
```

The port mapping means:

```
host port 8501 → container port 8501
```

13. Interview Answer Template

Short answer

I developed a laptop price prediction system using supervised regression. I cleaned and transformed laptop specifications, engineered PPI and storage features, grouped CPU, GPU, and operating system categories, and applied log transformation to the price target. I used OneHotEncoder and a ColumnTransformer inside a Scikit-learn Pipeline. I compared several algorithms, including Random Forest, Gradient Boosting, XGBoost, Voting, and Stacking. XGBoost produced the highest recorded R^2 of approximately 0.8964 on my test split. I exported the final pipeline with Pickle, created a Streamlit interface, and added Docker support for reproducible deployment.

If asked about MAE

MAE is the average absolute difference between actual and predicted values. Since my target was log-transformed, the MAE shown in the notebook is in log-price units. For business reporting, I should inverse-transform the predictions and calculate MAE in rupees.

If asked about R^2

R^2 measures how much variation in the target is explained by the model compared with a mean-prediction baseline. My XGBoost test result was approximately 0.8964 on the log-transformed target. This is not the same as 89.64% price accuracy.

If asked why Docker

Docker packages the application code, dependencies, model files, and runtime into a container. This makes the application easier to run consistently across different systems.

14. Suggested Improvements

1. Use cross-validation rather than relying on one train/test split.
2. Calculate and report MAE and RMSE in rupees after inverse transformation.

3. Use a separate validation set or nested cross-validation for model selection.
4. Tune XGBoost and ensemble hyperparameters.
5. Add a prediction interval or confidence explanation.
6. Validate input values against the training feature ranges.
7. Add a `.dockerignore` file to exclude notebooks, caches, and unnecessary files from the Docker build context.
8. Pin package versions in `requirements.txt` for reproducibility.
9. Verify the saved pipeline before deployment.
10. Add unit tests for PPI calculation and input DataFrame construction.